Programming Project #4: Some Suggestions

Computer Networks I CSCI-471 and CSCI 598

Plan before you start coding:

A few things to think about before you start coding.

- How will you store IP addresses?
  You will probably want to manipulate IP addresses as 32-bit unsigned integers, while input and output need to be done in dotted decimal. You might store addresses as strings, converting to numeric data only when you need. Alternatively, you could store them as unsigned integers and use a function to convert them to dotted decimal anytime you want to print them. You might consider an IP address class, that includes the output stream operator.

- Design your routing table.
  The routing table will form the core of the program, and having solid design will make the rest of the process simpler and more enjoyable. First you should think about what information you need to make routing decisions and then decide how you will structure those data elements (a class or a struct seems like the best choices to me, but there are other options). Next you need to pick a data structure that will hold all your routes. We will only be dealing with a small number of routes so an array of pointers or a container from the standard library would both be good options. A more efficient structure would be a *Trie*.

  Once you know what you need to store and how you will store it, you need to decide what operations you need to perform. Adding a route to the table is an obvious need. Anytime I am working with a multi-element data structure I like to take the time to write a function named dump() that prints out the whole data structure so I can confirm it is working like I expect. You will not use it once the program is finished, but it is nice to have while debugging.

  The third major choice you need to make is how you will use the route table to select routes. In my design I choose to hide most of the heavy lifting inside a single function "select_route(ip)" that, when given an IP address, would return the route with the longest prefix that matched the IP address. You could also move some of that work elsewhere, having your routing table simply hold the routes while other functions perform the analysis.

- Routes to local vs remote networks.
  Another design decision will be how to deal with directly connected subnets verses routes that need to be forwarded to a next-hop router. One way to think about the design is that there are two decisions to be made for each IP address that needs to be routed (1) is the destination IP address in a directly connected subnet? If so, which interface do we use. (2) if the destination IP is not in a directly connected then which next-hop router will you use, and which local subnet is that next-hop router in. One way to approach it is to treat the two decisions separately. Another choice would be to put the local subnets directly in the routing table with a bogus next-hop IP address and adjust the output accordingly when printing the output.

DRAFT 0.1

One practical hint:

Making use of the regular expression library will make reading the interface configuration and routing tables much simpler. The regular expressions I used for this task are:

```
R"(\s*([a-zA-Z0-9]+)\s+(\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3})/(\d{1,2})\s*)"
R"(\s*(\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3})/(\d{1,2})\s+(\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3})\s*)"
```

DRAFT 0.1